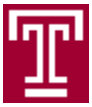

# Electrical and Computer Engineering
# ECE3622 Embedded Systems Design

## Introduction to Timer Based Measurement

Dennis Silage, PhD
*silage@temple.edu*

Consider the issues of range, accuracy, sampling rate, and drift described here before implementing timer based measurement. Input capture timers can be used to measure quantities from devices such as A/D converters and encoders. The issues to consider when using timers in this way include range, accuracy, sampling rate, and drift.

**Range**

Suppose that an application uses an embedded processor to control the speed of a DC motor. This motor may rotate at speeds in the range 1 to 3,000 revolutions per minute (RPM). To facilitate closed-loop control, an encoder is attached to the motor's shaft. The output of that encoder feeds a 16-bit input capture timer, where it can be read by the processor.

If the encoder produces 100 pulses with each revolution of the shaft, it will produce just one pulse every 600 ms at the motor's slowest speed. At the fastest speed, 3,000 RPM, the shaft encoder will produce a pulse every 200μs. Depending on the range of the input capture timer, some motor speeds may or may not be detected properly.

Typically, the reference clock to an input capture timer can be software-selected. The choice is generally between using the processor clock directly or prescaling (dividing) that clock by some power of two. If the processor clock is 4 MHz and the prescaler is set to 32, for example, the reference clock will be 125 kHz. But will this reference clock provide sufficient range?

With pulses occurring about every 200μs, as they do at the motor's fastest speed, the input capture register will contain a value of approximately 25 (125 kHz divided by 5,000 pulses/s) when it's read by the processor. Given this value and the above constants, the actual motor speed (3,000 RPM) can be calculated in software. At slower speeds, the count read by the processor will be higher and the calculated RPM smaller.

Something bad happens, though, with the same reference clock and the slowest motor speeds. Just below 2 RPM, the encoder pulses will occur so far apart that the 16-bit input capture timer will overflow between pulses. If an overflow happens, the software will read an incorrect timer value and miscalculate the motor's speed as a result—perhaps with dangerous consequences.

With a 125 kHz reference clock, the range of the 16-bit input capture timer in this example is insufficient to measure all possible motor speeds. Even if we enforced a 2 RPM lower limit on motor speed, timer overflow could still happen if the motor overshot this limit while slowing down.

The range of a timer must be considered any time you are using input capture and operating the timer near its full count value.

**Accuracy**

Continuing with the same example, each encoder pulse represents precisely 3.6 degrees (360˚/100 pulses) of shaft rotation. However, the rotational speed of the shaft cannot be calculated with such consistent accuracy across the full range. As the speed of rotation increases, the difficulty of detecting small changes in speed increases.

The problem is not the inaccuracy itself, but the fact that the accuracy of the calculated speed varies widely. A change from 25 to 26 counts, for example, drops the calculated speed from 3,000 to 2,885RPM in one fell swoop; no actual speed between those two values can ever be measured. At the other end of the range, though, changes as small as from 1.99995 to 2.00000RPM can be measured.

For real-world systems, the impact of such variations in accuracy must be thought through carefully. For example, a closed-loop control algorithm such as proportional integral derivative (PID), which is based on reacting to current and accumulated past errors, may swing more wildly around target speeds where accuracy is lowest. How can any one set of equations be expected to resolve inaccuracies ranging from 0.00005 to 115 RPM?

Similar considerations apply to timer outputs. If you are using an 8-bit timer to generate a pulse width modulation (PWM) signal, the output duty cycle can only be changed by one timer count, or 1 in 256. This results in a duty cycle resolution of 0.3%. However, this applies only if the timer is allowed to run a full 256 counts.

If you're using an 8-bit timer, but only using 100 counts for the PWM period, then one step is 1% of the total period—and that is the best resolution you can achieve. In an application where you vary both the PWM period and duty cycle, you need to be sure that the resolution at the fastest period (least number of timer counts per cycle) is adequate for the application.

A related issue arises when using an input capture timer to measure a length of time. Ambiguity in the counter values must be considered. For timing purposes, you can only assume that the counter value has an accuracy of +/- one timer count\

Given a timer with a 1 kHz reference clock (giving a 1 ms resolution) and input capture to measure the time between two events. Even if the first event occurs at count 51 and the second at count 52, they may not have occurred exactly 1 ms apart. As Figure 1

shows, the two events could be nearly simultaneous. Or, if they arrived at the beginning of count 51 and end of count 52, they could be almost a full 2ms apart.

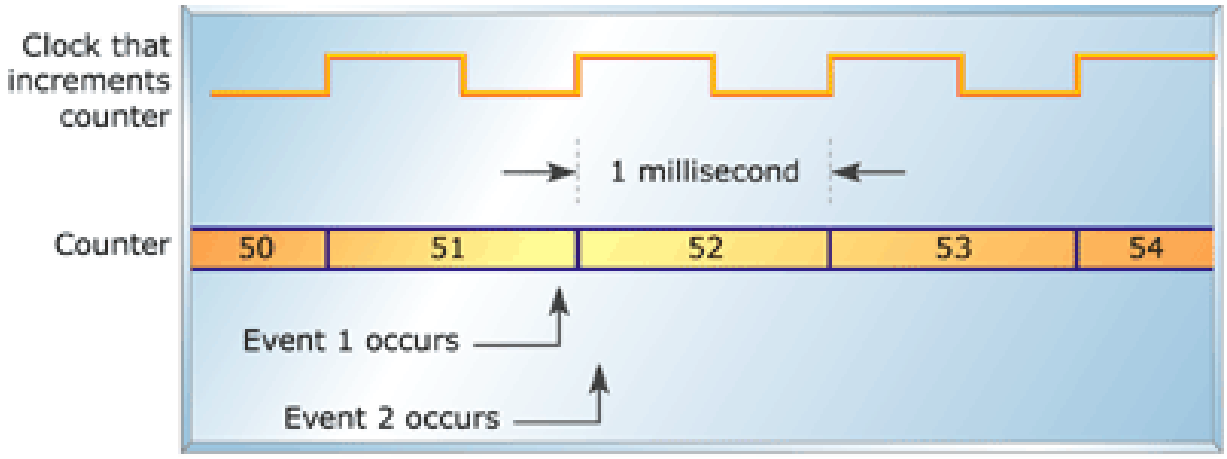


Figure 1: Timer ambiguity

**Sampling Rate**

Because it is dependent on encoder pulses to trigger input, our example system can measure the motor speed no faster than the encoder pulse rate. As the motor speed decreases, the time between new speed estimates falls. At 3,000 RPM, the sampling rate is 5 kHz; at 2 RPM, it falls to just 0.333Hz. If the control algorithm checks errors and tweaks motor speed only once per encoder pulse, then neither of those changes can be made more frequently than every 300 ms at 2 RPM. At the other end of the range, the software has just 200 µs to do its calculations and make an appropriate motor speed adjustment.

**Drift**

When you have two timers operating from different crystal oscillators, you must assume that the values in the timers will diverge over time. A typical scenario would involve timers on two different CPU boards in a multiprocessor system. If your system depends on the two timers remaining in sync, you must provide a synchronization mechanism between them. Any two clocks will always drift with respect to each other; without synchronization, timer values will ultimately diverge.

When working with timers, be sure to consider the issues of range, accuracy, sampling rates, and drift carefully. If you don't think about these issues as part of your design, something as seemingly simple as a counter/timer unit could bring your system design to its knees.